

LC 3568: Minimum Moves to Clean the Classroom

Authored by: Arpit Raj, LNMIIT Jaipur | Date: September 1, 2026

Copyright © 2026 Arpit Raj. All rights reserved.

Problem Breakdown

We are given an $(m \times n)$ grid with the following elements:

- 'S' → start
- 'L' → litter
- 'R' → area of rest energy
- '.' → empty
- 'X' → block
- **energy** → max energy of boy to begin cleaning

Moving $\uparrow \downarrow \leftarrow \rightarrow$ costs 1 unit energy.

If energy reaches 0, can continue **only if** on an 'R'.

Goal: Need to collect all litter, minimum moves.

First Thought & Approach

Count the number of litters in one iteration and then start level order BFS from 'S'.

So queue state will be: (row, col, energy, collected number of litters)

But this fails because:

Consider branching paths reaching point A:

Path 1: S → L1 → A

Path 2: S → L3 → A

Both have same row, col, energy, collected no of litter but **different states**.

Optimized State / Litter Mask

Hence we use `{row, col, energy, litter mask}`.

litter mask: Only 10 litters so we mark which litters are collected.

Additionally, since BFS and queue need to mark visited state, hence in the first iteration:

1. Find `sr, sc`
2. Since all litters are L, we give each litter a unique id i.e. `L1, L2, L3...`

```
So id[i][j] = cnt  
cnt++ when grid[i][j] == 'L'
```

So first litter gets id 0, means bit 0 is 1, i.e. `1000001`

```
fullmask = (1 << cnt) - 1
```

ALGORITHM

1. find `sr, sc`
2. give each litter a unique id
3. make queue and visited and initiate them
4. start moving
5. if `mask == fullmask`, return moves
6. continue if out of grid, negative eng, obstacle
7. update mask, continue if visited
8. mark visited, push to queue

C++ Implementation

```

class Solution {
public:
    int minMoves(vector<string>& classroom, int energy) {
        int n = classroom.size();
        int m = classroom[0].size();

        int sr = 0;
        int sc = 0;

        int cnt = 0;
        vector<vector<int>>> id(n, vector<int>(m, -1));
        for(int i = 0; i < n; i++) {
            for(int j = 0; j < m; j++) {
                if(classroom[i][j] == 'S') {
                    sr = i;
                    sc = j;
                }
                if(classroom[i][j] == 'L') {
                    id[i][j] = cnt;
                    cnt++;
                }
            }
        }

        // ek queue me {row,col,energy,mask}
        int fullmask = (1 << cnt) - 1;
        queue<pair<int, pair<int, pair<int, int>>>> q;
        vector<vector<vector<vector<bool>>>> visited(n + 1,
vector<vector<vector<bool>>>(m + 1, vector<vector<bool>>(energy + 1,
vector<bool>(1 << cnt, false))));

        q.push({sr, {sc, {energy, 0}}});
        visited[sr][sc][energy][0] = true;

        vector<pair<int, int>> dir = {{-1, 0}, {0, -1}, {1, 0}, {0, 1}};
        int moves = 0;

        while(!q.empty()) {
            int sz = q.size();

            while(sz--) {
                auto curr = q.front();
                q.pop();

                int r = curr.first;
                int c = curr.second.first;
                int eng = curr.second.second.first;
                int mask = curr.second.second.second;

                if(mask == fullmask) {
                    return moves;
                }

                for(auto &it : dir) {
                    int nr = r + it.first;
                    int nc = c + it.second;

```

```

        if(nr < 0 || nr >= n || nc < 0 || nc >= m) {
            continue;
        }

        if(classroom[nr][nc] == 'X') {
            continue;
        }

        int neweng = eng - 1;
        if(neweng < 0) {
            continue;
        }

        if(classroom[nr][nc] == 'R') {
            neweng = energy;
        }

        int newmask = mask;
        if(classroom[nr][nc] == 'L') {
            int idx = id[nr][nc];
            newmask = mask | (1 << idx);
        }

        if(visited[nr][nc][neweng][newmask]) {
            continue;
        }

        visited[nr][nc][neweng][newmask] = true;
        q.push({nr, {nc, {neweng, newmask}}});
    }
    moves++;
}
return -1;
}
};

```